설비 고장을 예측하는 AI 데이터 분석

NumPy 실습

센서 데이터 구조화와 배열 응용

posco

CONTENTS

목차

01 위치와 변화량 찾기 — argmax · argsort · diff · clip

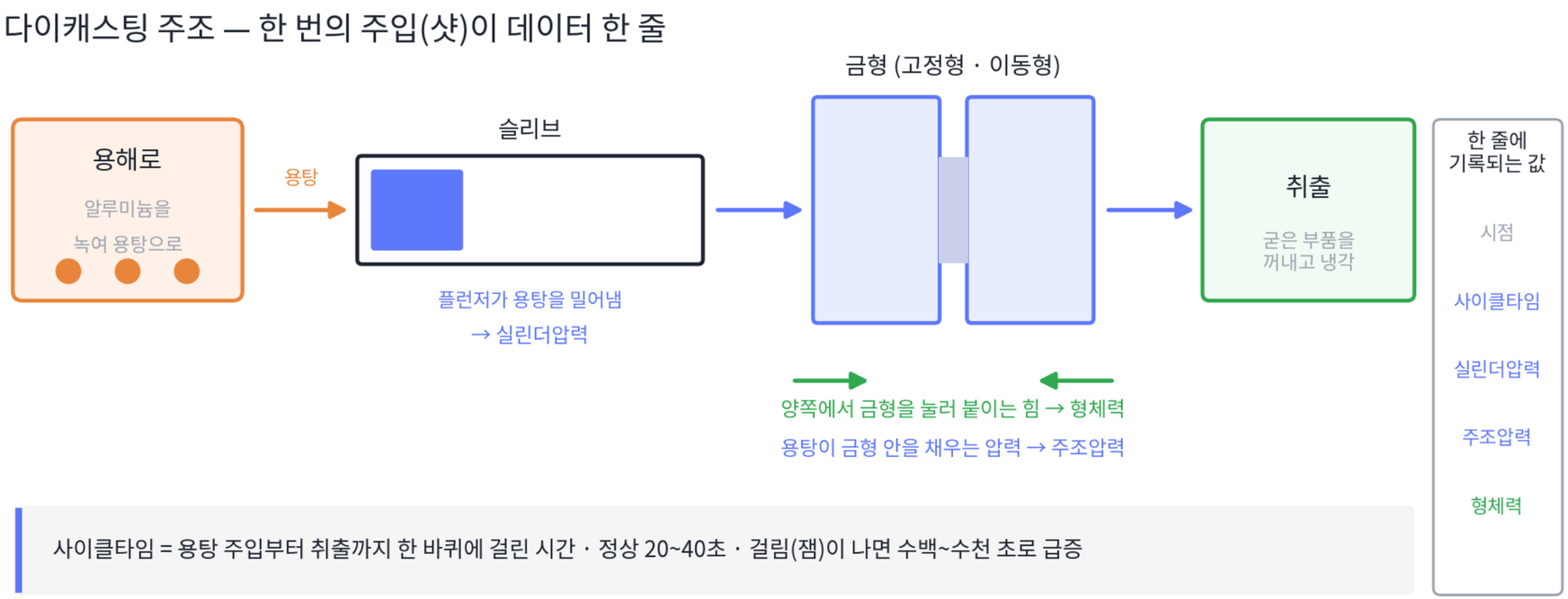
02 상태 라벨링과 배열 합치기 — where · unique · stack

03 종합 정리 — 도구를 하나의 흐름으로

posco

분석 대상 공정 — 다이캐스팅 주조

녹인 금속을 금형에 밀어 넣어 부품을 만드는 설비 · 한 번의 주입을 샷이라 부름



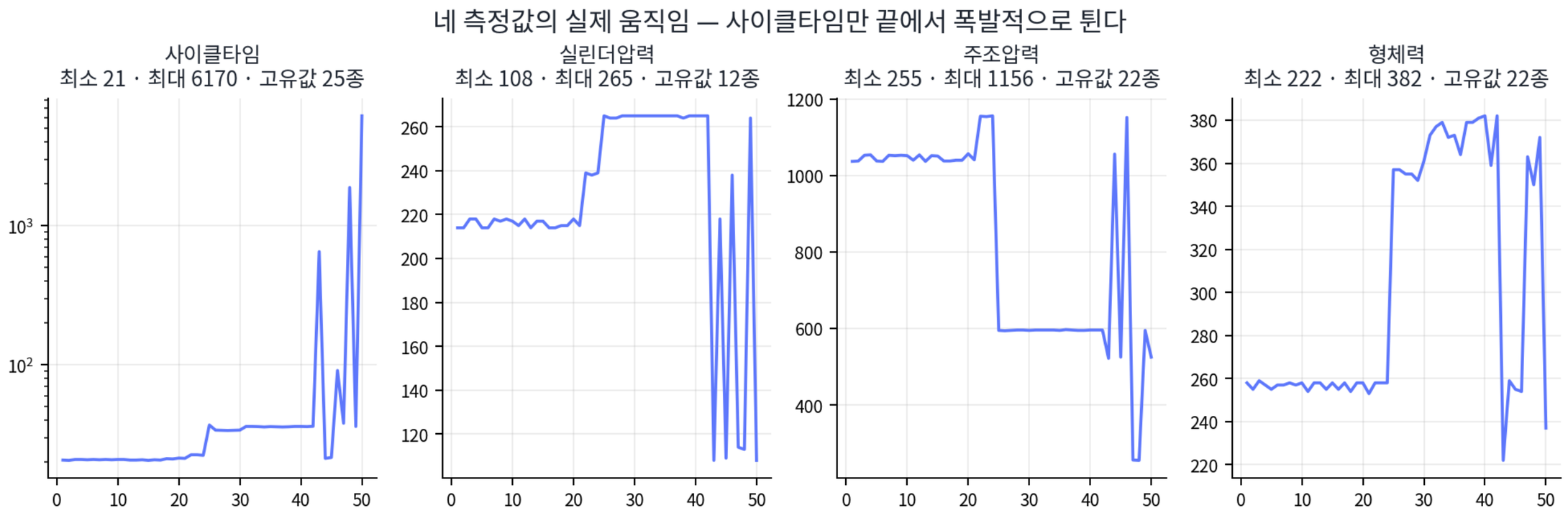
용해로에서 녹인 금속을 슬리브에 붓고, 플런저가 그것을 금형 안으로 밀어 넣음

한 번의 샷이 끝날 때마다 사이클타임·실린더압력·주조압력·형체력이 한 줄로 기록됨

출처: 주조(다이캐스팅) 설비 실데이터(KAMP 제조AI 데이터셋) 발췌 · 50샷 5열

네 측정값이 실제로 어떻게 움직이는가

무엇을 재는 값인지 알아야 결과를 해석할 수 있음 · 네 열의 실측 추이



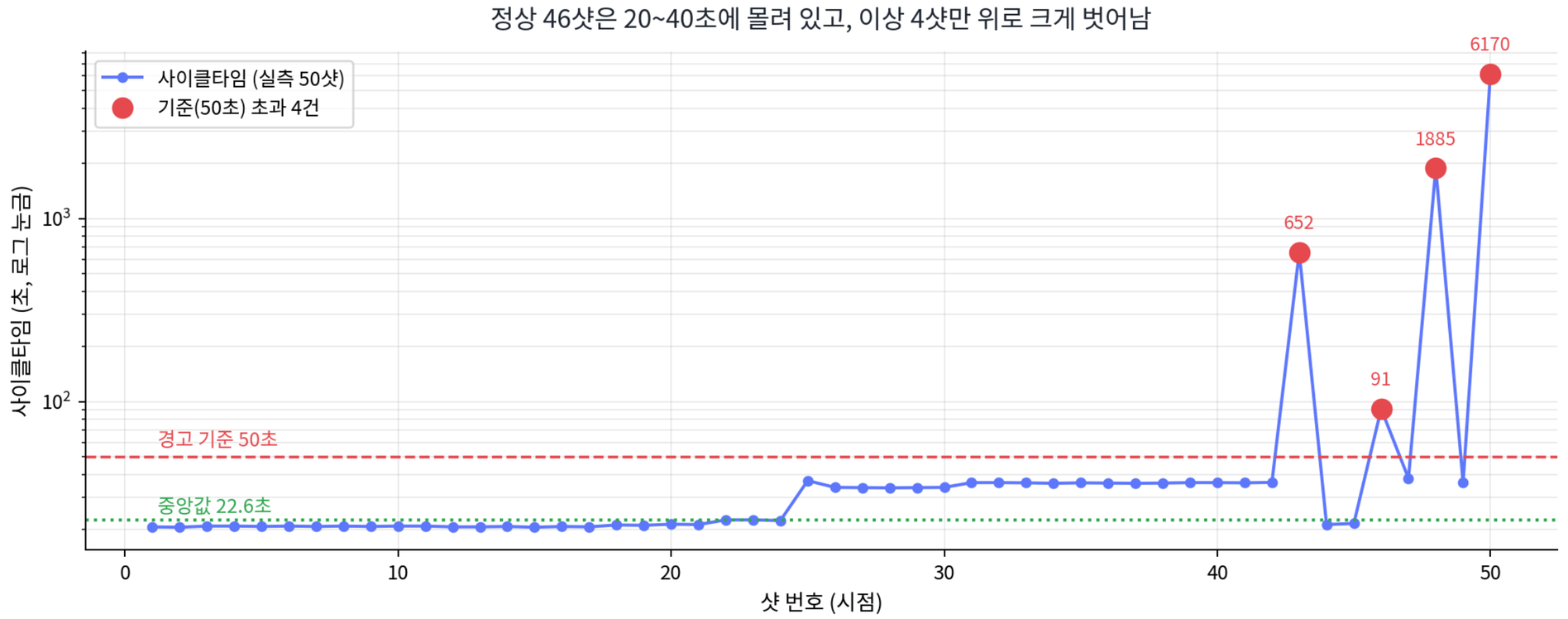
사이클타임 최대 6170.0초(중앙값의 약 273배) · 다른 센서도 각기 다른 범위로 변화

이번 실습은 사이클타임을 주요 지표로, 실린더압력을 추가 경고 조건으로 사용

실린더압력은 12종, 주조압력·형체력은 22종의 값만 나타남 — 설비가 몇 가지 조건으로만 운전됐다는 뜻

정상 샷과 잼 — 무엇이 다른가

사이클타임 50초 이상인 이상 후보 4건 · 이후 조건에 따라 경고 2건, 잼 2건으로 구분



50샷 중 46샷은 50초 미만, 4샷은 50초 이상 · 중앙값은 22.6초

50샷이면 눈으로 셀 수 있지만 하루치는 수천 샷 — 그래서 코드로 찾아야 함

50초는 이 실습에서 가정한 판정 기준 · 실제 현장 적용 전 설비별 검증 필요

배열 생성·속성

오늘 종일 쓸 도구라 손이 기억하게 다시 한 번 정리

01 · 배열 생성

리스트→배열 변환과 연속·균등·빈 배열

np.array로 리스트를 배열로
arange는 간격을 정해 값 생성
linspace는 같은 간격의 값 N개
zeros · ones는 0 · 1로 채운 배열

02 · 배열 속성

배열의 모양·차원 자료형 확인과 변환

shape는 모양
ndim은 차원 수
dtype은 자료형
astype으로 형 변환

03 · 벡터 연산

반복문 없이 한 줄로 전체 연산

반복문 없이 배열 전체에
한 번에 **사칙연산** 적용

데이터가 클수록 차이 극대화

수만 개 보정 한 줄로 끝나는 벡터 연산이 NumPy의 진짜 강점

posco

인덱싱·슬라이싱·조건 필터링

이상 센서값을 찾아내는 모든 작업의 출발점

01 · 인덱싱

위치 번호로
값 하나 꺼내기

대괄호에 위치 번호를 넣어 값 하나 꺼내기

0부터 시작, **-1**은 맨 뒤

02 · 슬라이싱

콜론으로
연속 구간 잘라내기

콜론으로 시작과 끝을 정해

연속 구간을 한 번에 잘라내기

03 · 불리언 인덱싱

True · False로
원하는 값만 골라내기

조건으로 만든 **True · False 배열** 로

원하는 값만 골라내기

불리언 인덱싱이 오늘 가장 자주 등장할 도구

posco

센서 분석의 4단계 흐름

담기 → 꺼내기 → 요약 → 판정, 네 단계 흐름

STEP 1

담기

센서값을 배열에 담는 단계

어제 배운 배열 만들기

→

STEP 2

꺼내기

원하는 부분을 꺼내고 조건으로 골라내는 단계

→

STEP 3

요약

수천 개 값을 평균·표준편차 같은 몇 개 숫자로 압축

→

STEP 4

판정

요약한 통계를 기준으로 이상한 값을 가려냄

argmax · diff · where

오늘은 배열을 활용해 위치·변화·기준 기반 판정까지, 다음 단원은 통계적 이상 탐지

posco

파일에서 배열로 불러오기(np.genfromtxt)

앞 단원의 loadtxt와 같은 일을 하되, 빈 칸을 만나도 멈추지 않고 값 없음으로 채움

CODE

```
import numpy as np

F = 'data/11_diecasting_series.csv'

ct = np.genfromtxt(F,
                   delimiter=',',    # 값을 나누는 기호
                   skip_header=1,    # 첫 줄(제목)은 건너뜀
                   usecols=1)        # 2번째 열 = 사이클타임
print(ct.shape, ct.dtype)
# 출력: (50,) float64

# 열 번호는 0부터
# 시점 0 · 사이클타임 1 · 실린더압력 2 · 주조압력 3 · 형체력 4
```

loadtxt는 건너뛸 줄을 skiprows, genfromtxt는 skip_header로 씀 · 열 번호는 둘 다 0부터

posco

01

위치와 변화량 찾기

argmax · argsort · diff · clip · cumsum

값이 아니라 자리를 찾는다

앞 단원은 「얼마인가」에 답했고, 이 단원은 「어디서 그랬는가」에 답한다

01 · 앞 단원에서 한 일

가장 큰 값은 얼마인가

최댓값 6170.0 을 구했음. 값 자체는 알았지만 그것이 몇 번째 샷이었는지는 따로 눈으로 찾아야 했음.

02 · 이 단원에서 할 일

그 값이 어디서 났는가

50번 샷이라는 답을 바로 얻음. 설비 점검은 「언제 무슨 일이 있었나」를 알아야 시작할 수 있음.

03 · 자리를 알면

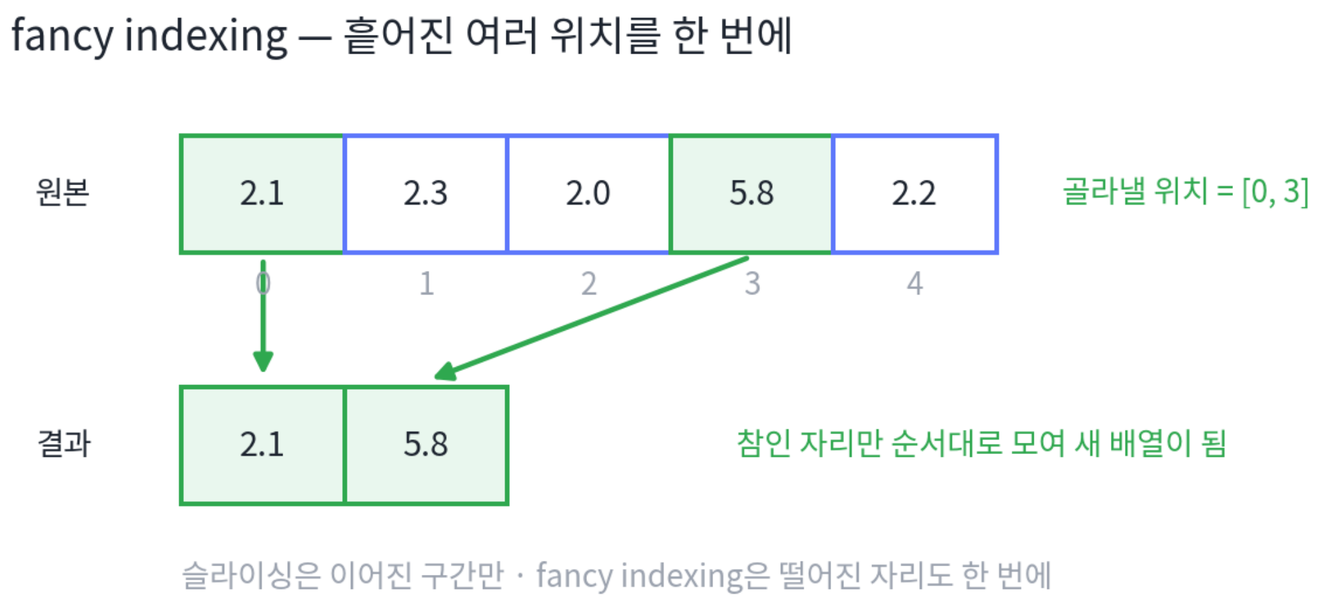
앞뒤를 함께 볼 수 있음

자리를 알면 그 시점 앞뒤의 다른 센서값도 같이 꺼내 볼 수 있음. 원인을 찾는 출발점이 여기서 만들어짐.

posco

여러 위치 한 번에 추출 — fancy indexing

흩어진 여러 자리의 값을 한 번에 꺼내는 도구



꺼내고 싶은 자리 번호를 묶음으로 만들어 대괄호에 넣으면 그 자리 값만 모인 새 배열이 생김

슬라이싱은 이어진 구간만 잘라낼 수 있지만, 이 방법은 떨어진 자리도 한 번에 꺼냄

결과는 원본과 분리된 새 배열 — 결과를 고쳐도 원본은 그대로

posco

fancy indexing — 핵심 코드

위치 목록을 리스트로 묶어 추출

CODE

```
vib = np.array([2.1, 2.3, 2.0, 5.8])
pick = np.array([0, 3])
print(vib[pick])
# 출력: [2.1 5.8]
```

정렬과 정렬 순서 — sort, argsort

값을 늘어놓은 결과와 그 값들이 원래 있던 자리 — 두 가지 정보가 따로 필요



sort는 값을 작은 것부터 늘어놓고, argsort는 그 순서에 해당하는 원래 자리 번호를 돌려줌

「가장 큰 값이 얼마인가」가 아니라 「언제 그랬는가」를 물을 때는 자리 번호가 필요

상위 세 개의 자리를 알고 싶으면 argsort 결과의 맨 뒤 세 개

posco

sort, argsort — 핵심 코드

sort로 값 정렬, argsort로 원래 자리 번호 확인

CODE

```
import numpy as np
vib = np.array([2.1, 6.1, 2.0, 5.8])
print(np.sort(vib))
# 출력: [2. 2.1 5.8 6.1]
print(np.argsort(vib))
# 출력: [2 0 3 1]
```

큰 값 상위 두 자리: np.argsort(vib)[-2:] → [3 1]

posco

최대·최소 발생 위치 — argmax, argmin

이상 분석의 첫 질문은 '가장 심했던 순간이 언제인가'

이상 분석의 첫 질문은 **'가장 심했던 순간이 언제인가'**

np.max는 값을, **np.argmax**는 그 값의 위치를 반환

찾은 위치를 사이클·온도 배열에 넣어 그 순간의 다른 정보까지 연결

> 이상 탐지에서 **'어디서'는 조치의 출발점**. 실무는 max보다 argmax가 빈번

posco

argmax, argmin — 핵심 코드

가장 큰 진동값의 위치를 찾아 사이클로 변환

CODE

```
import numpy as np
cycle = np.array([1, 2, 3, 4, 5])
vib = np.array([2.1, 2.3, 6.1, 2.0, 5.8])
pos = np.argmax(vib)
print(cycle[pos])
# 출력: 3
```

posco

실습 1. 최대 사이클타임 시점 찾기

사이클타임이 가장 긴 샷의 위치를 찾아 번호와 시간을 확인

목표

사이클타임의 최댓값이 나온 샷 번호와 사이클타임(초)을 출력

단계

① CSV에서 시점(0열)과 사이클타임(1열)을 각각 배열로 읽기

② 사이클타임 배열에서 가장 큰 값이 나온 인덱스 i 찾기

③ 인덱스 i를 시점 배열에 적용해 실제 샷 번호를 얻기

④ 같은 i를 사이클타임 배열에 넣어 실제 초 단위 값 꺼내기

⑤ 샷 번호와 해당 사이클타임을 출력하고 기준과 비교하기

실습 힌트 | 인덱스 i와 사이클 번호는 다름 · 50번 샷 = ct[49]

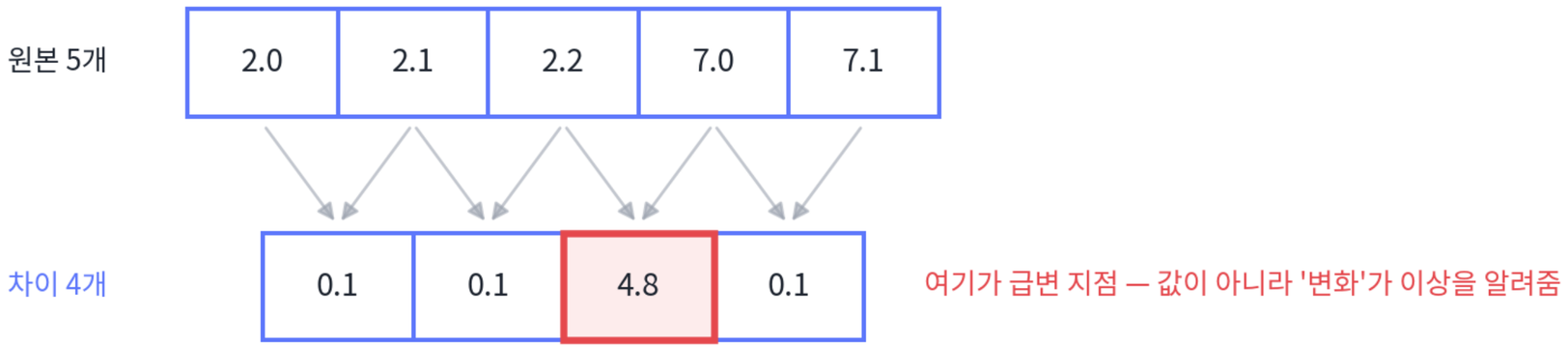
예상 결과

50번 샷에서 사이클타임 6170.0초 (배열 인덱스는 49)

인접 시점 변화량 — diff

값 자체보다 「얼마나 갑자기 변했나」가 더 강한 이상 신호

np.diff — 이웃한 값의 차이. 결과 길이가 하나 줄어든다



값 5개 → 차이 4개. 차이는 이웃 사이에서 생기므로 항상 하나가 짧다

이웃한 두 값의 차이를 구하므로 결과 개수는 원본보다 항상 하나 적음

k번째 변화량은 k번과 k+1번 사이에서 생긴 값 — 한 시점이 아니라 구간을 가리킴

급증도 급감도 똑같이 이상이므로 부호를 없앤 뒤 크기만 비교

posco

diff — 핵심 코드

변화량을 구하고 가장 급증한 위치 찾기

CODE

```
import numpy as np
vib = np.array([2.0, 2.1, 2.2, 7.0, 7.1])
change = np.diff(vib)
print(np.round(change, 1))
# 출력: [0.1 0.1 4.8 0.1]
print(np.argmax(change))
# 출력: 2
```

diff 결과는 원본보다 1 짧음

센서값 정리 — round, abs

이상 탐지는 방향 없이 거리만 보므로 abs가 결정적

np.round는 소수점 반올림, 보고·표시용 마지막 단계에서 사용

np.abs는 부호를 없애 변화의 크기만 비교

이상 탐지는 위·아래 어느 방향이든 거리만 보므로 abs가 중요

내일 z-score 이상 탐지에서 abs 재등장. **평균에서 떨어진 거리**를 잴 때 필수

round는 중간이 아니라 사람에게 보여주는 마지막 단계에서

round, abs — 핵심 코드

반올림과 절댓값으로 정리

CODE

```
import numpy as np
print(np.round(2.6666666, 2))
# 출력: 2.67

change = np.array([0.1, -4.8, 4.8, -0.1])
print(np.abs(change))
# 출력: [0.1 4.8 4.8 0.1]
```

변화량의 크기만 비교할 땐 abs가 핵심

posco

실습 2. 사이클타임 급변 구간 찾기

이웃한 두 시점 사이에서 값이 가장 크게 변한 구간을 찾기

목표

값이 가장 급격하게 변한 구간의 앞뒤 사이클 번호와 변화량을 출력

단계

① 시점과 사이클타임 두 열을 불러오기

② np.diff로 이웃한 사이클타임의 차이 구하기 (50개 → 49개)

③ np.abs로 급증·급감을 모두 양수 크기로 변환하기

④ 변환한 변화량의 최댓값 위치 j 구하기 (시점이 아닌 구간)

⑤ cyc[j]와 cyc[j+1], 해당 변화량(초)을 함께 출력

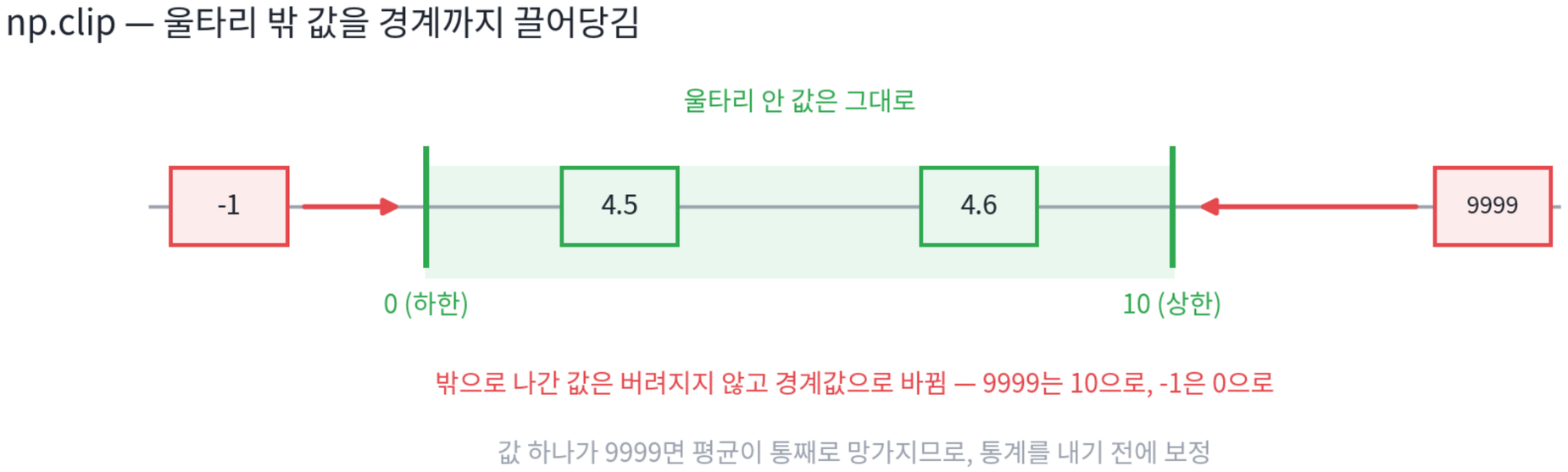
실습 힌트 | np.diff(ct) → np.abs(변화량) → np.argmax · 인접한 두 샷 번호 모두 출력

예상 결과

49번과 50번 사이에서 6133.9초만큼 변한 것으로 출력

센서값 범위 제한 보정 — clip

측정 오류로 튄 값은 눌러 두고 진짜 이상은 살려 두기



정해 둔 아래·위 경계를 벗어난 값은 버려지지 않고 경계값으로 바뀜

값 하나가 9999면 평균이 통째로 망가지므로 통계를 내기 전에 먼저 보정

경계를 어디에 둘지는 데이터가 아니라 그 설비를 아는 사람이 정하는 값

posco

clip — 핵심 코드

0~10 범위를 벗어난 값을 경계로 보정

CODE

```
import numpy as np
pres = np.array([4.5, 9999.0, 4.6, -1.0])
fixed = np.clip(pres, 0, 10)
print(fixed)
# 출력: [4.5 10. 4.6 0. ]
```

9999는 10으로, 음수는 0으로 보정

보정과 제거 — 어느 쪽을 쓸까

튄 값을 눌러 담을 것인가, 아예 빼 버릴 것인가

01 · 눌러 담기

줄 수가 그대로

경계까지 눌러 담으면 50샷이 50샷 그대로 남음. 시점이 어긋나지 않으므로 앞뒤 비교가 그대로 가능.

02 · 빼 버리기

줄 수가 줄어듦

튄 값을 빼면 그 시점이 통째로 사라져 앞뒤가 붙어 버림. 순서가 있는 데이터에서는 특히 조심해야 함.

03 · 판단 기준

측정 오류인가 실제 사건인가

센서가 잘못 읽어 튄 값이면 눌러 담아도 됨. 잼처럼 실제로 일어난 일이면 지우면 안 되고 표시해 두어야 함.

무엇을 할지 정하는 기준은 데이터가 아니라 설비를 아는 데서 나옴 — 처리 방법은 뒤 단원에서 자세히

posco

실습 3. 측정 오류값 범위 안으로 보정하기

범위를 벗어난 값 하나가 평균을 얼마나 망가뜨리는지 숫자로 확인

목표

오류가 섞인 상태와 보정한 뒤의 평균을 나란히 놓고 차이를 확인

단계

① CSV에서 실린더압력(2열)을 새 배열 cyl로 불러오기

② 메모리의 cyl[0]만 9999로 바꾸기 (CSV 파일은 수정하지 않음)

③ 오류가 섞인 상태의 평균을 구해 출력

④ np.clip(cyl, 100, 300)으로 보정 배열 fixed 만들기

⑤ 보정 뒤 평균을 출력하고 원래 평균(225.8)과 비교하기

실습 힌트 | cyl[0]=9999는 실습용 가짜 오류 · np.clip은 보정한 새 배열 반환

예상 결과

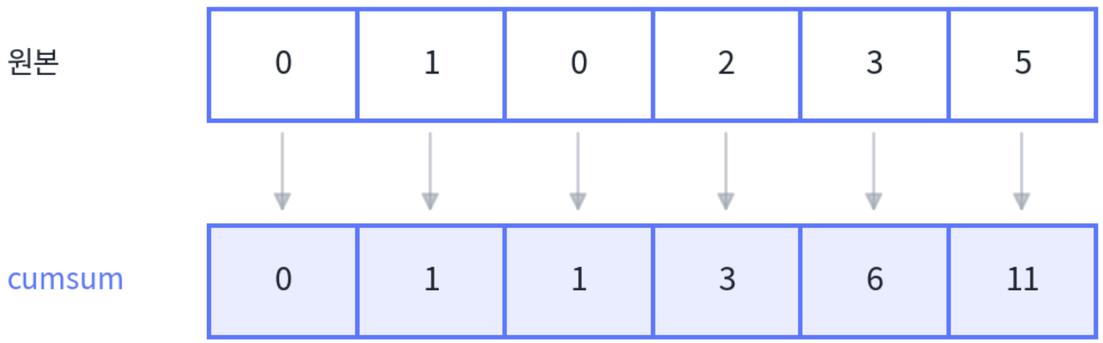
보정 전 평균은 421.5, 보정 뒤에는 227.5로 원래 수준 부근

posco

누적 합으로 추세 보기 — cumsum

순간값에 가려진 전체 흐름은 누적 곡선에서 매끄럽게 드러남

np.cumsum — 앞에서부터 더해 온 합계를 자리마다 남김



0 · 0+1 · 0+1+0 · … 처럼 앞쪽 값이 계속 누적됨. 누적 불량 수, 누적 가동 시간처럼 '지금까지 합계'를 볼 때 사용

자리마다 「거기까지의 합계」를 남기므로 결과 개수는 원본과 같음

누적 불량 수, 누적 가동 시간처럼 「지금까지 얼마나 쌓였나」를 볼 때 사용

입력값이 0 이상일 때 누적 합은 증가하거나 유지 · 누적 불량 수를 파악할 때 유용

posco

cumsum — 핵심 코드

사이클별 불량 개수의 누적 합

CODE

```
import numpy as np
defects = np.array([0, 1, 0, 2, 3, 5])
print(np.cumsum(defects))
# 출력: [0 1 1 3 6 11]
```

sum은 최종 합 하나, cumsum은 매 시점 누적값 전체

posco

위치·변화량 추적 흐름

센서값 → 변화량 → 크기 → 위치 → 적용, 다섯 단계 흐름

STEP 1

센서값

센서값 배열에서 시작.
진동·온도·압력 등
원본 측정값

`np.array`

→

STEP 2

변화량

diff로 인접 변화량을 구함

`np.diff`

→

STEP 3

크기

abs로 부호를 없애
크기만 남김

`np.abs`

→

STEP 4

위치

argmax로
가장 크게 변한 위치 탐색

`np.argmax`

→

STEP 5

적용

그 위치를
사이클·타 센서에 적용해
입체적 파악

`fancy indexing`

위치를 알면 사이클·다른 센서의 그때 상태까지 한 번에 연결

posco

02

상태 라벨링과 배열 합치기

where · unique · any · all · concatenate · stack

posco

다중 조건 라벨링 — np.where 심화

where의 진짜 힘은 판단 자동화

측정값을 **정상·경고 같은 상태 라벨로 자동 변환**

조건이 참이면 한 값, 거짓이면 다른 값을 채움

where를 겹쳐 쓰면 **3단계 이상 다단계 라벨링** 가능

> 기준을 바꾸고 싶으면 **코드의 숫자 하나만 고치면 전체가 다시 계산**

수천 개 측정값이 단번에 0과 1로 — 사람 판단 불필요

np.where 심화 — 핵심 코드

3단계 자동 라벨링 (5이상=2, 3이상=1, 그외=0)

CODE

```
import numpy as np
vib = np.array([2.1, 3.5, 2.0, 6.2])
label = np.where(vib >= 5, 2, np.where(vib >= 3, 1, 0))
print(label)
# 출력: [0 1 0 2]
```

바깥 where의 거짓 자리에 또 다른 where

posco

상태 라벨 종류 세기 — unique

분석 첫 단계는 '데이터에 어떤 상태가, 몇 개씩 있나' 확인

분석 첫 단계는 '데이터에 어떤 상태가, 몇 개씩 있나' 확인

np.unique는 값의 종류를 중복 없이 정리

return_counts 옵션으로 각 종류의 개수까지 함께 확인

설비 데이터에서 고장은 매우 드묾. **불균형을 모르면 엉뚱한 결론**

정상이 990개, 고장이 10개라면 크게 치우친 데이터

posco

unique — 핵심 코드

상태 종류와 개수를 한 번에 집계

CODE

```
import numpy as np
status = np.array([0, 0, 1, 0, 2, 1, 0])
kinds, counts = np.unique(status, return_counts=True)
print(kinds)
print(counts)
# 출력: [0 1 2] / [4 2 1]
```

return_counts를 켜야 개수가 함께 출력

posco

실습 4. 상태를 세 단계로 나누어 세기

사이클타임을 정상·경고·잼 세 단계로 나누고 각 단계가 몇 건인지 세기

목표

세 단계로 나눈 뒤 단계별 개수와 전체 대비 비율을 출력

단계

① 사이클타임 열을 불러오기

② 1000 이상=잼(2), 50 이상 1000 미만=경고(1), 나머지=정상(0)

③ 각 상태 번호(0·1·2)가 몇 개인지 세기

④ 상태별 개수를 전체 50개로 나눠 비율 구하기

⑤ 개수와 비율을 함께 출력해 경고 이상이 전체의 몇 할인지 확인

실습 힌트 | 정상=0 (50 미만) / 경고=1 (50~1000 미만) / 잼=2 (1000 이상)

예상 결과

정상 46 · 경고 2 · 잼 2, 비율은 0.92 · 0.04 · 0.04

posco

조건 충족 여부 판정 — any, all

현실의 안전 규칙을 코드로 옮기는 핵심 도구

01 · np.any

하나라도 만족하면
True

조건을 만족하는 값이 하나라도 있으면 True

경고 발생 판정

02 · np.all

모두 만족해야
True

모든 값이 조건을 만족해야 True

전체 안전 판정

03 · 활용

비교 → 판정
두 단계 구성

비교 조건으로 True · False 배열을 만든 뒤

any·all로 최종 판정

any 한 줄이면 수만 개 중 문제 여부를 즉시 확인 — 1차 선별 검사

any, all — 핵심 코드

하나라도 넘는지, 전부 안쪽인지 판정

CODE

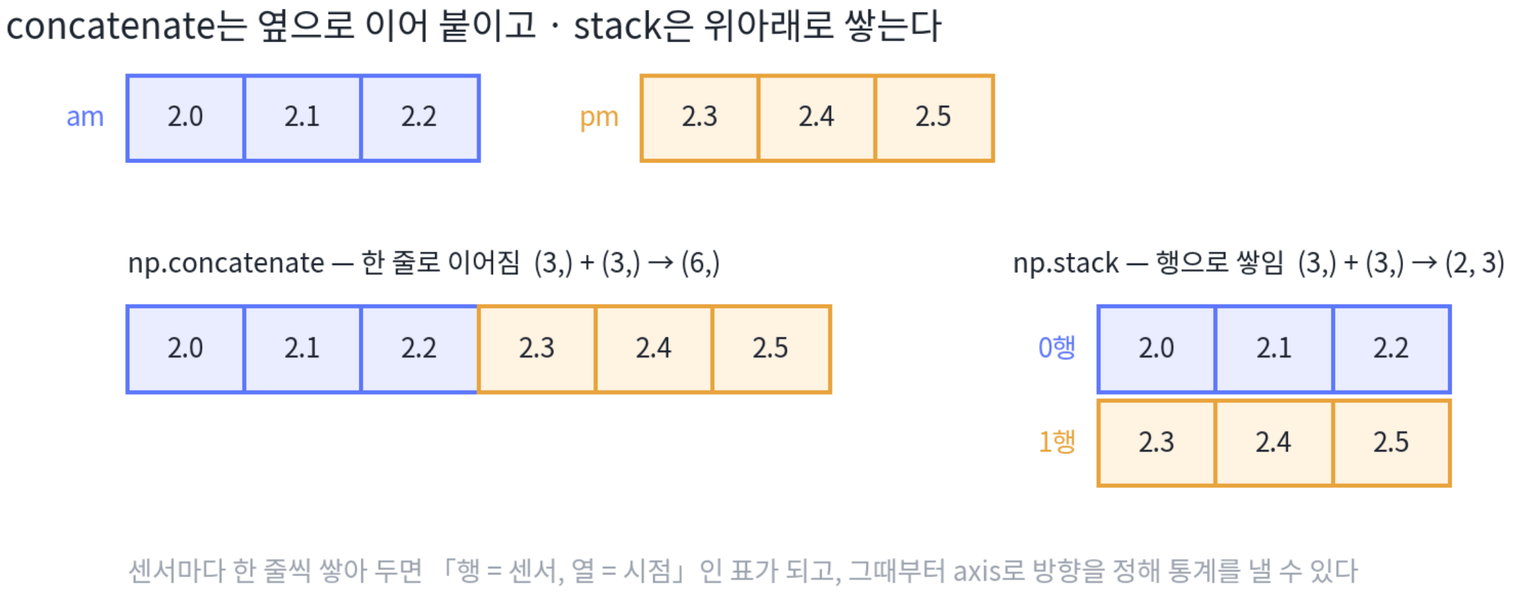
```
import numpy as np
vib = np.array([2.1, 2.3, 6.2, 2.0])
print(np.any(vib > 5))
# 출력: True
print(np.all(vib < 10))
# 출력: True
```

먼저 비교 조건으로 True·False 배열 생성 후 판정

posco

센서 배열 합치기 — concatenate, stack

한 줄로 잇는 concatenate · 행으로 쌓아 차원을 늘리는 stack



센서마다 한 줄씩 쌓아 두면 「행 = 센서, 열 = 시점」인 표가 되고, 그때부터 axis로 방향을 정해 통계를 낼 수 있다

같은 센서의 오전·오후처럼 이어 붙일 때는 concatenate, 서로 다른 센서를 나란히 볼 때는 stack

센서마다 한 줄씩 쌓으면 「행 = 센서, 열 = 시점」인 표가 되고 그때부터 방향을 정해 통계를 낼 수 있음

쌓으려면 길이가 같아야 함 — 센서마다 측정 개수가 다르면 먼저 길이를 맞춰야 함

posco

concatenate, stack — 핵심 코드

한 줄 연결과 2차원 쌓기 비교

CODE

```
import numpy as np
am = np.array([2.0, 2.1, 2.2])
pm = np.array([2.3, 2.4, 2.5])
print(np.concatenate([am, pm]))
# 출력: [2. 2.1 2.2 2.3 2.4 2.5]
print(np.stack([am, pm]).shape)
# 출력: (2, 3)
```

posco

2차원 배열 — 행은 센서, 열은 시점

실습 전에 이해하기 | 센서 4종 × 50번 샷을 하나의 표로 만들면 (4, 50)

센서별 행(row) / **시점별 열(column) →**

	1번	2번	3번	…	49번	50번
사이클타임	측정값	측정값	측정값	…	측정값	**6170.0**
실린더압력	측정값	측정값	측정값	…	측정값	측정값
주조압력	측정값	측정값	측정값	…	측정값	측정값
형체력	측정값	측정값	측정값	…	측정값	측정값

M[행, 열] · : 은 전체 선택 · -10: 은 마지막 10개 선택

코드로 확인

```
M = np.stack([...])
```

M.shape
→ (4, 50)

```
M[:, -10:]
```
→ (4, 10)

행 4개 유지, 최근 10열 선택

핵심: 각 행에는 센서 1종의 시간 흐름, 각 열에는 같은 샷의 센서 4종 값이 들어갑니다.

posco

실습 5. 다중 센서 행렬 만들고 구간 잘라내기

네 센서를 행으로 쌓아 표를 만들고 마지막 구간만 잘라내기

목표

행이 센서, 열이 시점인 표를 만든 뒤 최근 10 사이클 구간만 추출

단계

① 사이클타임·실린더압력·주조압력·형체력 네 열을 각각 불러오기

② np.stack으로 네 묶음을 쌓기: 행은 센서, 열은 시점

③ M.shape로 전체 행렬 크기 (4,50) 확인

④ 네 센서 행은 유지하고 마지막 10개 시점(열)만 잘라내기

⑤ recent.shape를 출력해 (4,10)인지 확인

실습 힌트 | M[행, 열] · 모든 행은 : / 마지막 10개 열은 -10: · 결과 (4,10)

예상 결과

전체는 4행 50열, 잘라낸 뒤는 4행 10열

posco

데이터 준비 흐름

합치기 → 보정 → 라벨링 → 집계, 네 단계 흐름

STEP 1

합치기

concatenate · stack으로
흩어진 데이터를 모음

`concatenate · stack`

→

STEP 2

보정

clip으로 명백한 측정 오류를
정상 범위로 누름

`clip`

→

STEP 3

라벨링

where로 정상·경고·고장을
자동으로 매김

`np.where`

→

STEP 4

집계

unique로
전체 중 경고 개수를 요약

`np.unique`

순서가 바뀌면 — 보정 전 통계는 오류값에 휘둘림

03

종합 정리

오늘 배운 도구를 하나의 흐름으로 엮기

posco

실습 6. 이상 시점 한 번에 정리하기

최대 시점 · 급변 구간 · 경고 비율 · 상위 세 시점을 한 흐름으로 정리

목표

오늘 배운 도구를 순서대로 이어 붙여 한 화면짜리 점검 결과 만들기

단계

① 시점 · 사이클타임 · 실린더압력 세 열을 불러오기

② 사이클타임이 가장 컸던 시점과 그때의 값 찾기

③ 이웃한 값의 변화량이 가장 컸던 구간 찾기

④ 사이클타임이 50 이상이거나 실린더압력이 150 아래인 시점을 경고로 보고 비율 구하기

⑤ 사이클타임이 큰 순서로 상위 세 시점을 뽑아 함께 출력

실습 힌트 | OR는 | / AND는 & · 각 조건은 괄호로 · True 평균은 경고 비율

예상 결과

최대 50번 6170.0 · 급변 49→50 · 경고 비율 0.12 · 상위 세 시점 50 · 48 · 43

CLOSING

posco

오늘 배운 도구가 놓인 자리

위치를 찾고 · 변화를 재고 · 상태를 매기고 · 표로 묶는다

1

위치를 찾는다 — argmax · argsort

「얼마나」가 아니라 「언제」에 답해야 정비 이력을 찾아볼 수 있음

2

변화를 잰다 — diff · abs · clip

값보다 변화가 강한 신호. 재기 전에 명백한 오류값은 눌러 둠

3

상태를 매기고 묶는다 — where · unique · stack

숫자를 등급으로 바꾸고 센서를 표로 쌓는 것이 표 도구를 만나기 전 마지막 준비

다음 단원에서는 같은 일을 훨씬 짧은 코드로 하는 표 도구를 만남

CLOSING

여기서 배운 것이 다음 단원에서는

도구가 바뀌어도 하는 일은 같음 — 훨씬 짧은 코드가 될 뿐

01 · 열 번호에서 열 이름으로

번호를 세지 않아도 됨

지금은 몇 번째 열인지 세어 넣었음. 다음 단원에서는 이름을 그대로 적음. 열 순서가 바뀌어도 안전해짐.

02 · 숫자만에서 글자까지

한 표에 함께

배열에는 숫자만 담을 수 있어 시각과 상태는 빼고 읽었음. 다음 단원의 표는 글자 열도 같은 표에 담음.

03 · 하나씩에서 한 번에

통계를 한 줄로

평균과 최대와 중앙값을 열마다 따로 구했음. 다음 단원에서는 숫자 열 전부에 대해 한 줄로 끝남.

오늘 익힌 흐름 — 자리를 찾고 변화를 재고 상태를 매긴다 — 은 그대로 이어짐